

TP Java B3 FS — « Arena Legends » : le simulateur de tournoi

Sep 29, 2026 · @Ali

Présentation

Vous allez coder **Arena Legends**, un simulateur de tournoi de combattants en console, 100 % Java pur (pas de framework). Chaque partie du TP débloque une brique du jeu : à la fin, 8 combattants s'affrontent dans un tournoi à élimination directe et le programme affiche le champion.

Objectifs : maîtriser les conditions, les boucles, les tableaux, les listes (`ArrayList`), l'encapsulation et l'héritage, puis les combiner dans un vrai petit projet orienté objet.

Règle d'or : le code doit compiler à chaque commit. Un code qui ne compile pas vaut 0 sur la partie concernée.

Partie 1 — Conditions et boucles : le menu et les dés

Créez une classe `Main` avec un menu en console qui tourne en boucle jusqu'à ce que l'utilisateur tape `0`.

```
=== ARENA LEGENDS ===
1. Lancer un dé
2. Calculer un rang
3. Test de coup critique
0. Quitter
```

1. **Menu** : utilisez une boucle `do...while` et un `switch`. Toute saisie invalide (lettre, nombre hors menu) affiche un message d'erreur sans faire planter le programme.
2. **Lancer un dé** : demandez le nombre de faces (entre 4 et 20, redemandez tant que c'est faux avec un `while`), puis tirez un nombre avec `java.util.Random`.
3. **Calculer un rang** : à partir d'un nombre de points saisi, affichez le rang avec des `if / else if` : moins de 100 = Bronze, 100 à 499 = Argent, 500 à 1499 = Or, 1500 et plus = Légende. Un nombre négatif est refusé.
4. **Test de coup critique** : simulez 10 000 attaques avec une boucle `for`. Chaque attaque a 15 % de chance d'être critique. Affichez le nombre de critiques et le pourcentage réel obtenu.

5. **Piège** : affichez la plus longue série de critiques consécutifs obtenue pendant les 10 000 tirages. Interdiction de stocker les tirages : une seule passe, deux variables.

Question README : pourquoi un `do...while` est-il plus adapté qu'un `while` pour un menu ?

Partie 2 — Tableaux : l'historique de l'arène

Créez une classe `StatsArene` avec uniquement des méthodes `static`. On vous donne les scores de 12 matchs :

```
int[] scores = {42, 87, 15, 99, 63, 87, 5, 71, 99, 34, 50, 28};
```

1. Écrivez `moyenne(int[] t)`, `max(int[] t)` et `min(int[] t)` **sans** utiliser `Arrays` ni aucune méthode toute faite.
2. Écrivez `trierDecroissant(int[] t)` qui trie le tableau avec un **tri à bulles** codé à la main. Ajoutez un compteur d'échanges et arrêtez le tri dès qu'un passage ne fait aucun échange.
3. Écrivez `sansDoublons(int[] t)` qui retourne un **nouveau tableau** de la bonne taille exacte, sans doublons (ici 10 cases). Le tableau d'origine ne doit pas être modifié.
4. **Tableau 2D** : l'arène est une grille `char[8][8]` remplie de `'.'`. Placez aléatoirement 6 obstacles `'#'` (jamais deux au même endroit) et 2 combattants `'A'` et `'B'`. Affichez la grille proprement.
5. **Piège** : écrivez `distance(char[][] grille)` qui retourne la distance de Manhattan entre `A` et `B` ($|x1 - x2| + |y1 - y2|$), en parcourant la grille une seule fois.

Question README : quelle est la différence entre copier un tableau avec `int[] b = a;` et le recopier case par case ? Montrez-le avec un exemple.

Partie 3 — Encapsulation : le combattant blindé

Créez une classe `Combattant`. Aucun attribut ne doit être `public`. On doit être **incapable** de mettre le combattant dans un état absurde depuis l'extérieur.

Attribut	Type	Règle à garantir
<code>nom</code>	<code>String</code>	Non null, 3 à 15 caractères, jamais modifiable après création
<code>pvMax</code>	<code>int</code>	Entre 50 et 300, fixé à la création
<code>pv</code>	<code>int</code>	Toujours entre 0 et <code>pvMax</code>

Attribut	Type	Règle à garantir
attaque	int	Entre 5 et 50
defense	int	Entre 0 et 30
nbCombattants	static int	Compte les combattants créés, lecture seule

1. Écrivez le constructeur. Une valeur invalide lève une `IllegalArgumentException` avec un message clair (ex. : "attaque doit être entre 5 et 50, reçu : 70").
2. Écrivez les getters utiles. **Pas de setter pour pv** : la vie ne change que via `subirDegats(int d)` et `soigner(int s)`.
3. `subirDegats(int d)` : les dégâts réels valent `d - defense`, avec un minimum de 1. Les PV ne descendent jamais sous 0.
4. `soigner(int s)` : impossible de dépasser `pvMax`, et un combattant K.O. (0 PV) ne peut plus être soigné.
5. Ajoutez `estK0()` et redéfinissez `toString()` pour afficher par exemple `Kaelen [72/120 PV] ATK 18 DEF 6`.
6. **Piège** : ajoutez un attribut `int[] historiqueDegats` (les 5 derniers dégâts reçus) et un getter. Prouvez dans le `main` que votre getter ne permet **pas** de modifier l'historique depuis l'extérieur (copie défensive).

Question README : pourquoi un setter `setPv(int pv)` casserait-il l'encapsulation même s'il vérifie les bornes ?

Partie 4 — Héritage et polymorphisme : les classes de héros

Rendez `Combattant` **abstraite** et ajoutez la méthode abstraite `int attaquer(Combattant cible)`, qui inflige des dégâts à la cible et retourne les dégâts envoyés. Créez trois sous-classes :

Classe	Attribut propre	Comportement de attaquer
Guerrier	rage (0 à 100)	Dégâts = attaque. Chaque attaque ajoute 20 de rage ; à 100, dégâts doublés puis rage remise à 0
Mage	mana (départ 100)	Sort à 30 de mana : dégâts = attaque × 2 et ignore la défense. Sans mana : attaque ÷ 2 et récupère 15 mana
Voleur	esquive (en %, 10 à 40)	Dégâts = attaque, 25 % de chance de frapper deux fois. Redéfinit <code>subirDegats</code> : peut esquiver totalement

1. Chaque constructeur appelle `super(...)`. Les attributs de `Combattant` restent `private` : les sous-classes passent par les getters ou des méthodes `protected`.
2. Pour le Mage, ajoutez dans `Combattant` une méthode `protected void subirDegatsBruts(int d)` qui ignore la défense. Justifiez le choix de `protected` dans le README.
3. `Voleur` redéfinit `subirDegats` avec `@Override` et appelle `super.subirDegats(d)` quand il n'esquive pas.
4. Ajoutez la méthode abstraite `String getClasse()` et intégrez-la dans le `toString()` de `Combattant`, sans le redéfinir dans les sous-classes.
5. **Piège** : créez une classe `Paladin` extends `Guerrier` qui se soigne de 10 % de ses dégâts infligés à chaque attaque. Réutilisez la logique de rage du Guerrier sans copier-coller une seule ligne.

Question README : dans `Combattant c = new Mage(...); c.attaquer(x);`, quelle méthode est appelée et pourquoi ? Expliquez la différence entre type déclaré et type réel.

Partie 5 — Listes : le moteur de tournoi

Créez une classe `Tournoi` qui contient une `private ArrayList<Combattant> participants`. C'est ici que tout se rejoint.

1. `inscrire(Combattant c)` : refuse les doublons de nom (comparaison insensible à la casse) et bloque au-delà de 8 inscrits. Retourne `true` ou `false`.
2. `desinscrire(String nom)` : retire le combattant. Attention : supprimer dans une liste pendant un `for-each` provoque une `ConcurrentModificationException`. Utilisez un `Iterator` ou une boucle `for` inversée.

3. `duel(Combattant a, Combattant b)` : les deux s'attaquent à tour de rôle (celui qui a la meilleure attaque commence) jusqu'au K.O. Au-delà de 50 tours, le vainqueur est celui qui a le plus de PV en pourcentage. Chaque tour s'affiche en console.
4. `lancer()` : tournoi à élimination directe. Mélangez la liste, faites s'affronter les combattants deux par deux, placez les vainqueurs dans une **nouvelle** liste, soignez-les entièrement (pensez à votre règle sur les K.O.) et recommencez jusqu'à ce qu'il n'en reste qu'un.
5. `classement()` : retourne une `ArrayList<Combattant>` triée par nombre de victoires décroissant (ajoutez un compteur de victoires encapsulé). Tri codé à la main, pas de `Collections.sort`.
6. `statsParClasse()` : affiche pour Guerrier, Mage, Voleur et Paladin le nombre de victoires cumulées, en utilisant `getClasse()` et non `instanceof`.
7. **Piège** : `getParticipants()` ne doit pas permettre de modifier la liste depuis l'extérieur. Testez que `tournoi.getParticipants().clear()` n'a aucun effet sur le tournoi.

Test final attendu dans le `main` : 8 combattants (au moins 2 de chaque classe), le tournoi complet affiché, puis le champion, le classement et les stats par classe. Lancez-le 100 fois sans affichage et indiquez dans le README quelle classe gagne le plus souvent. Est-ce équilibré ?